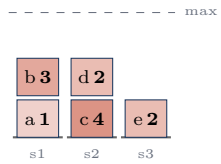
Assignment 7: Pre-marshalling in PDDL

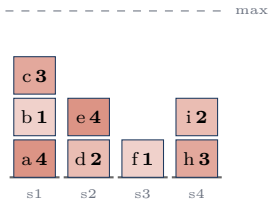
Reading: Russell & Norvig, Ch. 11. Files: `premarshalling-domain.pddl`, `premarshalling-bay1.pddl` (to complete), `planner.ipynb` (optional).

This assignment is one entry in your **portfolio**. Hand in your two completed PDDL files and about 1–2 pages of explanations. Parts A and B are required; Part C is optional.

The problem, as in Assignment 5: a bay has stacks of at most H containers. Every container has a group number; group 1 leaves the terminal first. A container is *blocking* if a container of a smaller group is somewhere below it. The crane takes the top container of a stack and puts it on top of a stack that is not full. Goal: no container is blocking.



Bay 1 (required): 3 stacks, height 3



Bay 2 (optional): 4 stacks, height 4

Each box shows the container's name and its **group**; stacks are drawn bottom to top.

The model. This time you don't write a search algorithm: you describe the problem in PDDL and let a planner do the work. The skeleton `premarshalling-domain.pddl` explains the model in its comments: every stack has a *base*; the crane picks a container up and puts it down, as in Blocks World; *tiers* `t0`, `t1`, ... enforce the height limit; groups are objects ordered by the static predicate `(leq g1 g2)`, and every base belongs to a special group `gtop` that leaves last. A place is `well-placed` if nothing in it or below it is blocking, and the goal is that every container is well-placed. Read the whole skeleton before you start: types, predicates and the action `pick-up` are given.

Part A – The domain (required)

A1. Write the action `put-down-well` (TODO 1): the crane puts the container it holds on a place `?to` such that the container is well-placed afterwards.

A2. Write the action `put-down` (TODO 2): the crane puts the container on any place whose stack is not full.

A3. Explain your `put-down-well`: why is each precondition needed, and why is the container then really well-placed? Why must `pick-up` delete `well-placed`?

A4. `put-down` never adds `well-placed`, even when the container happens to land in a good place. Why does this not make the model wrong, i.e. why are no solutions lost?

Part B – The problem (required)

B1. Complete the initial state in `premarshalling-bay1.pddl` for bay 1 (TODO 3). Which containers are well-placed at the start, and why?

B2. Write the goal (TODO 4). Why is it not a list of `on` atoms, as in Blocks World?

B3. Find a shortest plan for bay 1 by hand and write it as a sequence of actions with their arguments. How many plan steps, and how many crane moves?

Part C – Run a planner (optional)

Use `planner.ipynb` in JupyterLite, as in the lab (no installation needed), or Pyperplan on your own computer.

C1. Solve bay 1. Does the planner find your plan from B3, or another one of the same length? If it reports an error or finds no plan, what was wrong?

C2. Write a problem file for bay 2 and run `compare`. What do the heuristics do to the number of expanded nodes, the time and the plan length? Which results are guaranteed optimal?

C3. Try a bay from Assignment 5 (by hand, or with a small script that writes the PDDL). Compare with your own A* from Assignment 5, if you have it: what does a domain-specific heuristic buy you, and what does a general planner buy you?

Tips. Pyperplan accepts STRIPS with types only: no negative preconditions, no `or`, `forall` or conditional effects; express everything with the given predicates. Typical errors: a missing bracket, a predicate with the wrong number of arguments, a variable without `?`, an object of the wrong type. Check your actions against the self-check questions from the lecture: can a container be put on a full stack, on itself, or picked up while the crane is busy? Remember that JupyterLite keeps files in your browser only: keep your own copies.